

第1章 数据类型与变量

建议学时:3-5 小时 | 难度:★★☆☆☆ | 读者背景:已掌握 C 语言

🎯 本章学习目标

- 掌握 C++ 全部内建类型,说出与 C 类型的——对应及差异(long 在 Windows 上是 32 位!)
- 理解四种初始化写法与 auto 推导,搞清楚"未初始化"在 C++ 里意味着什么
- 理解 const 与 constexpr 的本质区别,用编译期常量替代 #define
- 掌握 static_cast 等显式转换,理解列表初始化为何拒绝窄化
- 会用 std::string、std::vector、std::map 替代 C 的 char[] 与裸数组,会用 enum class
- C 对照:知道 C 中未初始化、有符号溢出等 UB 在 C++ 中同样存在,学会主动规避

💡 从 C 到 C++:本章主题如何迁移

如果你能熟练写出 `struct Node { int data; struct Node* next; }` 这样的 C 代码,那么 C++ 的数据类型对你来说九成是熟面孔:int、double、char、指针,字节数与语义几乎不变。C++ 是 C 的超集,语法向下兼容,但它在类型体系上做了三件大事。

第一,类型更安全。C 的 enum 常量泄漏到全局、能隐式变成整数;C++11 的 enum class 把名字关进作用域,拒绝隐式转换。C 的 (int*)&d 想怎么强转就怎么强转;C++ 提供 static_cast 等四种语义明确的转换,把"窄化转换"列为编译错误。

第二,标准库补齐了 C 的短板。C 的 char[] 要靠 strlen/strcpy 手工维护长度,数组传参退化为指针;C++ 的 std::string 自己管理内存与长度, std::vector 是自动扩容的"安全数组", std::map 直接提供键值对。生产环境里,C 风格字符串与裸数组几乎只出现在遗留代码中。

第三,编译期能力大幅增强。C 的 #define MAX 100 只是文本替换;C++ 的 constexpr 变量与函数在编译期完成计算与类型检查。读完本章,你应该默认使用现代写法。

📦 前置知识自查

- C 的内建类型:char/int/long/float/double 的字节数与取值范围
- sizeof 运算符、#define 宏常量、typedef
- 字符串字面量与转义序列、strcpy/strlen/strcmp 家族
- 数组与指针的关系、数组传参退化
- 隐式类型转换与整型提升、C 风格强转 (int)x

🚀 本章学习路线

1. 第一阶段(熟悉):通读 §1.1–§1.4,重点类型对照表与初始化规则,编译运行"类型大小实测"程序
2. 第二阶段(重点):§1.5–§1.7 是最值钱的部分——转换规则、`std::string`、`std::vector`,边读边把 C 写法翻译成 C++
3. 第三阶段(收尾):§1.8 枚举与别名,做自测题与章末实验
4. 验收标准:能默写"类型对照表",能说清 `int a;` 与 `int a{};` 的区别,能徒手写出 `std::string` + `std::vector` 的小程序

本章知识导图

- §1.1 类型总览:C→C++ 类型对照;bool;char 三兄弟;固定宽度整型
- §1.2 变量:四种初始化、默认 vs 值初始化、auto 推导
- §1.3 字面量:进制、后缀、分隔符、原始字符串、编码前缀
- §1.4 常量:const 的局限、constexpr、constexpr/constinit
- §1.5 类型转换:隐式规则、列表初始化禁窄化、四种 cast
- §1.6 字符串:char[] 痛点 → std::string 常用 API
- §1.7 复合类型:std::array、std::vector、std::map、结构化绑定
- §1.8 枚举与别名:enum class、using

§1.1 类型总览:从 C 类型表到 C++ 类型表

C++ 的内建类型与 C 几乎一一对应。注意:标准不规定精确字节数,只保证大小关系,下表以最常见的 64 位平台为例,程序应随时用 `sizeof` 查询。

C 的写法	C++ 的写法	典型字节(64 位)	说明
int	int	4	语义完全相同
long	long	Win:4;Linux:8	最大陷阱:MSVC 恒 32 位
unsigned	unsigned	4	等价 unsigned int
float/double	float/double	4/8	IEEE 754
无	bool	1	_Bool 现代化身
char	char	1	符号性实现定义
size_t	std::size_t	8	有符号对应 ptrdiff_t
long long	long long	8	跨平台 64 位用它

1.1.1 bool:C 没有的"真正的布尔"

C 直到 C99 才有 `_Bool`,且本质上是整数。C++ 的 `bool` 是独立类型,只有 `true` / `false` 两个值,任何非零值赋给它都会变成 `true`。`sizeof(bool)` 通常是 1,不是 1 位。生产习惯:逻辑判断用 `bool`,不要用 `int` 充当标志位。

1.1.2 char 的三大陷阱

- 符号性未定义: `char` 是 `signed` 还是 `unsigned` 由编译器决定,跨平台读写二进制必须显式用 `signed char / unsigned char` 存字节
- `char` 不是"字符":汉字在 UTF-8 下占 3 字节, `char` 存不下一个汉字;文本处理用 `std::string`
- `char` 参与算术先提升为 `int`: `char c = 100 + 100;` 在有符号 `char` 上可能溢出,与 C 完全一致

1.1.3 固定宽度整型:C++11 的礼物

`<cstdint>` 提供 `std::int8_t/16_t/32_t/64_t` 及无符号版本。生产建议:网络协议、二进制文件、位域计算一律用固定宽度类型,普通计数用 `int` 或 `std::size_t`。

⚠ 易错点 1:long 在 Windows 与 Linux 上字节数不同

Windows(MSVC,LLP64)上 `long` 是 4 字节,64 位 Linux(GCC,LP64)上是 8 字节。同样的 `long x = 3000000000L;` 在 Windows 上溢出、在 Linux 上正常。生产规则:需要确定 64 位时写 `long long` 或 `std::int64_t`,不要写 `long`。

示例 1-1 内建类型大小实测

```
#include <iostream>
#include <cstdint>

int main() {
    std::cout << "long      = " << sizeof(long)      << "\n"; // Windows 上是 4!
    std::cout << "long long = " << sizeof(long long) << "\n";
    std::cout << "size_t    = " << sizeof(std::size_t) << "\n";
    std::cout << "int64_t   = " << sizeof(std::int64_t) << "\n";
    return 0;
}
```

§1.2 变量:声明、初始化与 auto

1.2.1 四种初始化写法

C 只有 `int x = 0;` 一种写法,C++ 有四种,含义各不相同:

C 的写法	C++ 的写法	含义
<code>int x = 0;</code>	<code>int x = 0;</code>	拷贝初始化,兼容 C
无	<code>int x(0);</code>	直接初始化
无	<code>int x{0};</code>	列表初始化,禁止窄化,推荐
无	<code>int x{};</code>	值初始化,归零

生产建议:标量与结构体一律用 `{}`,同时杜绝"忘记初始化"与"窄化截断"两类问题。

1.2.2 未初始化:C 的 UB 文化在 C++ 中的延续

C 里 `int x; printf("%d", x);` 是未定义行为(UB)。C++ 规矩相同:内置类型默认初始化后值不确定,读取即 UB;类类型调用默认构造函数;而 `int x{};` 是值初始化,保证得到 0。全局与静态变量无论哪种写法都零初始化。

⚠ 易错点 2:auto 推导会剥掉 const 和引用

`const int a = 1; auto b = a;` 中 `b` 是 `int`, 不是 `const int` ——`auto` 按值推导, 拷贝了一份。同理 `int& r = x; auto r2 = r;` 得到 `int`。需要保留修饰时写 `const auto& c = a;`。

示例 1-2 变量初始化与 auto 推导

```
#include <iostream>

struct Student {           // 类类型:有默认构造函数
    int id;                 // 无初始化器:值不确定(等价于 C)
    double score = 0.0;    // C++11 默认成员初始化器
};

int main() {
    int a;                  // 默认初始化:值不确定,别读
    int b = 0;              // 拷贝初始化
    int d{0};               // 列表初始化
    int e{};                // 值初始化:确定得到 0

    auto f = 3.14;          // double
    const auto h = 42;      // const int
    std::cout << b << d << e << f << h << "\n";

    Student s1;             // id 不确定,score 是 0.0
    Student s2{};           // 值初始化:id 也归零,推荐
    std::cout << s1.score << s2.id << "\n";
    return 0;
}
```

1.2.3 auto:类型推导的两面性

`auto` 不是“动态类型”,变量在编译期就已确定类型。它最适合类型名冗长时(`std::map<std::string, std::vector<int>>::iterator` 写起来痛不欲生)。生产约定:公共接口显式写类型,局部变量放心用 `auto`,推导可读性差时宁可写全。

§1.3 字面量:C++ 的扩展

示例 1-3 字面量全家福

```
#include <iostream>

int main() {
    int dec = 42;           // 十进制
    int hex = 0x2A;         // 十六进制
    int bin = 0b101010;     // C++14 起支持二进制
    int sep = 1'000'000;     // C++14 数字分隔符
    long long ll = 42LL;     // long long 后缀

    double pi = 3.14159;    // 默认 double
    float f = 3.14f;        // float 后缀 f

    char ch = 'A';
    auto s = "hello";        // const char[6], 以 '\0' 结尾
    auto raw = R"(C:\Users\dev\x)"; // 原始字符串: 免转义
    std::cout << dec << hex << bin << sep << ll << pi << f
              << ch << s << raw << "\n";
    return 0;
}
```

原始字符串是生产利器:写正则、JSON、Windows 路径时免双重转义。语法 `R"(内容)"`, 内容里出现 `)` 时可加定界符 `R"xx(内容)xx"`。编码前缀: `u8` 是 UTF-8(`char8_t`, C++20 起), `u` 是 UTF-16, `U` 是 UTF-32。生产规范:文件存 UTF-8、字符串写 `u8` 前缀、全程用 `std::string` 承载。

§1.4 常量:const、constexpr 与编译期计算

1.4.1 const 的局限与 C++ 的修正

C 的 `const int N = 100; int a[N];` 在标准 C 里不能保证编译通过;C 的 `const` 本质是"运行期只读变量"。C++ 修正了这一点:C++ 的 `const` 整型常量是编译期常量,可用于数组大小、模板参数。但 `const` 只在编译期检查,不保证对象不在别处被改(`const int*` 可指向可变对象)。

1.4.2 constexpr:真正的编译期常量

C++11 引入 `constexpr`, C++14 起 `constexpr` 函数里允许循环与分支。它比 `const` 强:值必须在编译期确定。生产替代关系: `#define MAX 100` → `constexpr int kMax = 100;` 后者有类型、有作用域、不污染预处理器。

示例 1-4 `const` 与 `constexpr` 的分工

```
#include <iostream>

constexpr int kMaxItems = 100;           // 替代 #define MAX_ITEMS 100
constexpr int kSquare(int n) { return n * n; } // constexpr 函数

int main() {
    int arr[kMaxItems];                  // 编译期大小,可以
    int arr2[kSquare(10)];               // constexpr 函数也能定数组大小

    int n = 0;
    std::cin >> n;
    const int runtime = n;               // const:运行期只读
    // constexpr int bad = n;           // 编译错误:n 不是编译期常量
    constexpr int kFixed = 2 + 3;       // 正确
    return 0;
}
```

1.4.3 C++20 的 consteval 与 constexpr

constexpr 强制函数只能在编译期求值; **constexpr** 要求静态变量用常量初始化。生产上 **constexpr** 已足够 90% 场景。

§1.5 类型转换:隐式、显式与窄化

1.5.1 隐式转换:与 C 完全一致的规则

整型提升、有符号/无符号混合、double 吸收 float——C 的规则原样继承,C 的坑也全部保留:int 与 unsigned 比较时 int 被转成无符号;有符号溢出仍是 UB。写作时主动避免混用有符号与无符号, `std::vector<T>::size()` 返回 `size_t`, 与 int 比较前先想清楚。

1.5.2 列表初始化拒绝窄化

`int x{300.5};` 是编译错误——double 到 int 是窄化转换。这是 C++ 对 C 最有价值的改进之一:把危险的隐式转换从运行期事故变成编译期错误。而 `int x = 300.5;` 仍能编译(兼容 C),只是截断成 300。

1.5.3 四种显式转换

转换	用途
<code>static_cast</code>	有明确定义的转换,日常 95% 用它
<code>const_cast</code>	去掉 const;能不用就不用
<code>reinterpret_cast</code>	重新解释二进制位;危险,审慎
<code>dynamic_cast</code>	多态运行时向下转型(第6章),有 RTTI 开销

C 风格强转 `(int)x` 是组合拳,编译器不拦任何错误。生产规范:代码中禁止 C 风格强转,统一用四种具名转换,便于 grep 与 review。

示例 1-5 类型转换的正确姿势

```
#include <iostream>

int main() {
    int i = 300;
    char c = static_cast<char>(i);      // 显式转换:我知道可能截断
    double d = i;                      // 隐式:int → double,无损
    i = static_cast<int>(d);            // double → int,丢弃小数

    // int x{300.5};                    // 编译错误:拒绝窄化
    int y{300};                        // 正确:无损放入

    unsigned u = 1u;                   // 有符号/无符号混用陷阱
    int s = -1;
    std::cout << (s < u) << "\n";      // 输出 0!s 被转成 unsigned

    char a = 100, b = 100;              // 整型提升:char 先转 int 再运算
    char sum = static_cast<char>(a + b); // a+b 是 int,等于 200,再转回

    std::cout << sum << y << "\n";
    return 0;
}
```

§1.6 字符串:从 char[] 到 std::string

💡 C 字符串的三大痛点,C++ 的三种解法

C 里字符串是"以 `\0` 结尾的 char 数组": `strcpy` 不知道目标容量,拼长了缓冲区溢出; `strlen` 每次 $O(n)$ 扫描;很多人错误地拿 `==` 去比指针。 `std::string` 一次性解决:内存自动扩容、 `size()` 是 $O(1)$ 、 `==` 比较内容。生产铁律:禁止用 C 字符串函数操作裸缓冲区。

C 的写法	C++ 的写法	差异
<code>strlen</code>	<code>s.size()</code>	$O(1)$ vs $O(n)$
<code>strcmp</code>	<code>a == b</code>	直接比内容
<code>strcpy</code>	<code>s2 = s;</code>	自动管容量
<code>strcat</code>	<code>s2 += s;</code>	自动扩容
<code>strstr</code>	<code>s.find(t)</code>	返回下标,失败 npos

示例 1-6 std::string 常用操作

```

#include <iostream>
#include <string>

int main() {
    std::string s = "hello";
    s += " world";           // 拼接,自动管理容量
    std::cout << s.size() << "\n"; // 0(1),不是 strlen 的 O(n)
    std::cout << (s == "hello world") << "\n"; // 内容比较,不是指针!
    std::cout << s.substr(0, 5) << "\n";       // 子串 "hello"
    std::cout << s.find("world") << "\n";       // 下标 6;找不到返回 npos
    s.replace(6, 5, "C++"); // 替换 [6, 11)
    const char* cstr = s.c_str(); // 与 C 接口互操作:只读 C 字符串

    std::string msg = "count=" + std::to_string(42); // 数值转字符串
    int n = std::stoi("42"); // 字符串转数值

    // 只读视图 string_view(C++17):零拷贝,只做函数参数
    std::string_view sv = "只读数据";
    std::cout << n << msg << cstr << sv.size() << "\n";
    return 0;
}

```

工程建议:编码与性能

编码:现代项目的字符串策略是"内部统一 UTF-8 的 `std::string`,只在 I/O 边界做编码转换"。不要用 `std::wstring` 处理业务数据——Windows 上是 UTF-16,Linux 上是 UTF-32,行为不一致。性能:小字符串优化 (SSO)让小串(约 15 字符内)不分配堆内存;拼接大量小串用 `reserve()` 预分配。

§1.7 复合类型入门:array、vector 与 map

1.7.1 `std::array`:知道长度的固定数组

C 的数组传参退化为指针,函数里拿不到长度——这是 C 数组最大的设计缺陷。`std::array<T, N>` 是值语义的数组,能 `size()`、`at()` 越界检查、能整体拷贝赋值,性能与 C 数组相同。

1.7.2 `std::vector`:生产环境的默认容器

`std::vector<T>` 对应 C 里"malloc + 手工 realloc"的场景,但自动管理:满了翻倍扩容,析构自动释放。注意:扩容会使迭代器与引用失效;删除元素用 `erase-remove` 惯用法(第3章详述)。

示例 1-7 array、vector 与 map 入门


```

#include <iostream>
#include <array>
#include <vector>
#include <map>

int main() {
    // at() 越界抛异常;[] 越界是 UB(和 C 一样)
    std::array<int, 5> a{1, 2, 3, 4, 5};
    std::cout << a.size() << a.at(0) << "\n";

    std::vector<int> v;           // 动态数组,自动扩容
    v.push_back(10);
    v.push_back(20);
    v.emplace_back(30);         // 原地构造,免拷贝
    v[1] = 99;
    std::cout << v.size() << v.front() << v.back() << "\n";

    for (int x : v) {           // 范围 for:现代遍历正统写法
        std::cout << x << " ";   // 10 99 30
    }

    std::map<std::string, int> scores; // 红黑树,按键有序
    scores["张三"] = 90;           // 键不存在则插入
    std::cout << scores["张三"] << scores.size() << "\n";

    for (const auto& [name, score] : scores) { // 结构化绑定(C++17)
        std::cout << name << ":" << score << " ";
    }
    return 0;
}

```

⚠ 易错点 3:vector 越界与迭代器失效

对 `std::vector` 用 `[]` 越界访问是未定义行为——它不会检查,和 C 数组一样可能悄悄踩坏内存。需要安全访问用 `v.at(i)` (越界抛 `std::out_of_range`),需要性能用 `[]` 并先保证下标合法。另外,`push_back` 导致扩容时,之前保存的迭代器、指针、引用全部失效——这是 C 里不存在的“新坑”,第3章专门讲。

§1.8 枚举与类型别名

1.8.1 从 C 的 enum 到 enum class

C 的 `enum Color { RED };` 把 `RED` 泄漏到全局作用域,且枚举值能隐式转 `int`、`int` 也能隐式转回枚举——类型形同虚设。C++11 的 `enum class` 是真正的强类型:名字在作用域内,不隐式转 `int`,还能指定底层类型(默认 `int`,可改成 `char` 省空间)。

示例 1-8 强类型枚举与类型别名

```

#include <iostream>
#include <vector>

// C 的写法:enum Color { RED, GREEN, BLUE }; // 名字泄漏进全局
// C++11 强类型枚举:作用域隔离,不隐式转整数
enum class Color : int { RED, GREEN, BLUE };
enum class Status : char { OK = 0, WARN = 1, ERR = 2 };

int main() {
    Color c = Color::GREEN;
    // int n = c; // 编译错误:不能隐式转整数
    int n = static_cast<int>(c); // 必须显式转换

    // C 用 typedef,现代 C++ 用 using(可用于模板,更可读)
    using Byte = unsigned char;
    using IntList = std::vector<int>;

    IntList v{1, 2, 3};
    Byte b = 200;
    std::cout << static_cast<int>(b) << v.size() << "\n";
    return 0;
}

```

1.8.2 别名不是新类型

`using Id = int;` 只是别名, `Id` 与 `int` 完全互通。若需要"防混淆的新类型"(学号与工号都是 `int` 但不应混用),需要包装结构体等手段——第14章"新类型模式"会讲。

本章速查表

主题	速查内容
固定 64 位整型	<code>std::int64_t(<cstdint>)</code> ;跨平台别用 <code>long</code>
布尔	<code>bool</code> : <code>true/false</code> , <code>sizeof</code> 为 1
char 三兄弟	<code>char</code> 符号性未定义; <code>signed/unsigned char</code> 存字节
未初始化	内置类型默认初始化值不确定(UB); <code>int x{};</code> 得 0
推荐初始化	一律 <code>{}</code> ;拒绝窄化
auto 陷阱	<code>auto</code> 剥掉 <code>const</code> /引用;要保留写 <code>const auto&</code>
字面量扩展	<code>0b1010</code> 、 <code>1'000'000</code> 、 <code>R"(...)"</code>
常量	<code>#define</code> → <code>constexpr</code> ;运行期只读用 <code>const</code>
窄化转换	<code>int x{300.5};</code> 编译错误;= 赋值截断
具名转换	<code>static_cast</code> 为主; <code>dynamic_cast</code> 多态下行
字符串	<code>std::string</code> : <code>size()</code> $O(1)$ 、 <code>==</code> 比内容、 <code>c_str()</code> 对接 C
动态数组	<code>std::vector</code> : <code>push_back/emplace_back</code> 、 <code>at()</code> 越界检查
键值对	<code>std::map</code> :有序、 <code>operator[]</code> 不存在即插入
强类型枚举	<code>enum class</code> :作用域隔离、显式转 <code>int</code>

最小必会清单

- 能说出 `int/long/long long/size_t` 在你平台上的字节数,并解释为何用固定宽度类型
- 能解释 `int a;`、`int a{;};`、`int a{0};` 三者的行为差异
- 能说清 `auto` 何时丢掉 `const` 与引用,并正确写出 `const auto&`
- 能写出二进制字面量、数字分隔符、原始字符串的语法
- 能区分 `const` 与 `constexpr`,知道数组大小需要哪种
- 能解释"列表初始化拒绝窄化"并举例,能说出四种具名转换的用途
- 能徒手写出 `std::string` 的拼接、查找、子串、替换代码
- 能用 `std::vector` 与 `std::map` 完成增删改查,知道 `at()` 与 `[]` 的区别
- 能写出 `enum class` 并说出它与 C 枚举的三个区别

自测题

Q 在 64 位 Windows 上, `sizeof(long)` 与 `sizeof(long long)` 分别是多少?跨平台需要 64 位整数时应该写什么?

✓ 参考答案

Windows(LLP64)上 `long` 是 4 字节, `long long` 是 8 字节。需要确定 64 位时写 `std::int64_t`。

Q 下面哪些行是编译错误?为什么? `double d = 3.14; int a{3.14}; int b = 3.14; int c{3};`

✓ 参考答案

`int a{3.14};` 是编译错误——列表初始化拒绝窄化。`int b = 3.14;` 能编译但截断为 3(兼容 C)。`int c{3};` 与 `double d = 3.14;` 正确。

Q `unsigned u = 1; int s = -1;` 表达式 `s < u` 的结果是什么?请解释。

✓ 参考答案

结果是 `false(0)`。混合比较时 `int` 被转换为 `unsigned`, -1 变成 4294967295,不小于 1。生产代码应避免有符号/无符号混比。

Q 写出下列代码中 a 到 d 的推导类型: `int x = 5; const int& r = x; auto a = r; auto& b = r; const auto c = r; auto* d = &x;`

✓ 参考答案

a 是 `int` (拷贝,剥掉引用与 `const`); b 是 `const int&`; c 是 `const int`; d 是 `int*`。

Q 用 `std::string` 把 `"apple,banana,cherry"` 按逗号切分,打印每段并统计总字符数。写出完整程序。

✅ 参考答案

```
#include <iostream>
#include <string>
#include <vector>
#include <sstream>

int main() {
    std::string line = "apple,banana,cherry";
    std::vector<std::string> parts;
    std::string token;
    std::istringstream ss(line);
    while (std::getline(ss, token, ',')) {
        parts.push_back(token);
    }
    std::size_t total = 0;
    for (const auto& p : parts) {
        std::cout << p << "\n";
        total += p.size();
    }
    std::cout << "总字符数: " << total << "\n";
    return 0;
}
```

章末实验题:学生信息登记与成绩统计

实验 1:学生信息登记与成绩统计

实验目标:写一个命令程序,登记学生信息并统计成绩,把全章的类型、初始化、常量、转换、字符串、容器、枚举、别名知识点全部用上。

实验要求:

- 任务 1:定义 `struct Student`,含学号(int)、姓名(std::string)、性别(bool)、班级(char)、成绩(double)(覆盖:结构体、内置类型、字符串)
- 任务 2:用 `constexpr` 定义及格线等常量,不用 `#define` (覆盖:编译期常量)
- 任务 3:用 `enum class Grade` 表示成绩等级,函数按分数返回等级(覆盖:强类型枚举)
- 任务 4:用 `std::vector` 存学生、`std::map` 统计各等级人数(覆盖:容器、range-for、结构化绑定)
- 任务 5:一律用 `{}` 初始化;算平均分、及格率时显式 `static_cast` (覆盖:列表初始化、类型转换)
- 任务 6:格式化输出总数、平均分、及格率、各等级人数(覆盖:to_string、setprecision)
- 任务 7:用 `auto` / `const auto&` 推导遍历变量,说出每处的推导类型(覆盖:auto 规则)

实验环境:任意 C++17 编译器(Visual Studio 2022 / MinGW g++ / clang++)。编译命令:g++ -std=c++17 main.cpp -o main

第一步 搭骨架:头文件、常量、枚举、结构体,编译通过后再加函数

第二步 `classify()` 函数与登记代码, `push_back` 传花括号初始化的临时对象

第三步 计与格式化输出;平均分先把 `size()` 转成 `double` 再除,避免整数除法;运行验证与手算一致

✅ 参考答案要点(第一部分:常量、枚举、结构体与函数)

```
#include <iostream>
#include <string>
#include <vector>
#include <map>
#include <iomanip>    // 输出格式化

// 任务 2:constexpr 常量,替代 #define
constexpr double kPassScore = 60.0;

// 任务 3:强类型枚举
enum class Grade { EXCELLENT, GOOD, PASS, FAIL };

Grade classify(double score) {           // 按分数返回等级
    if (score >= 90.0) return Grade::EXCELLENT;
    if (score >= 75.0) return Grade::GOOD;
    if (score >= kPassScore) return Grade::PASS;
    return Grade::FAIL;
}

// 任务 1:学生结构体
struct Student {
    int id;                // 学号
    std::string name;      // 姓名
    bool male;             // 性别:true=男
    char classCode;        // 班级代号 'A'/'B'
    double score;          // 成绩
};

// 附加任务:按学号查找,返回下标,未找到返回 -1
int findById(const std::vector<Student>& list, int targetId) {
    for (std::size_t i = 0; i < list.size(); ++i) {
        if (list[i].id == targetId) return static_cast<int>(i);
    }
    return -1;
}
```

✅ 参考答案要点(第二部分:main 主流程)

```
int main() {
    // 任务 4:vector 登记学生,{}-列表初始化
    std::vector<Student> students;
    students.push_back({1001, "张三", true, 'A', 92.5});
    students.push_back({1002, "李四", false, 'A', 78.0});
    students.push_back({1003, "王五", true, 'B', 55.5});
    students.push_back({1004, "赵六", false, 'B', 87.0});

    // 任务 5:统计,显式转换避免整数除法
    std::map<Grade, int> gradeCount;    // 等级 -> 人数
    double sum = 0.0;
    int passed = 0;
    for (const auto& s : students) {    // 任务 7:const auto& 避免拷贝
        sum += s.score;
        if (s.score >= kPassScore) ++passed;
        ++gradeCount[classify(s.score)];
    }
    double avg = sum / static_cast<double>(students.size());
    double passRate = static_cast<double>(passed) / students.size() * 100.0;

    // 任务 6:格式化输出
    std::cout << std::fixed << std::setprecision(2);
    std::cout << "总人数: " << students.size() << "\n";
    std::cout << "平均分: " << avg << "\n";
    std::cout << "及格率: " << passRate << "%\n";
    for (const auto& [grade, cnt] : gradeCount) {    // 结构化绑定遍历 map
        std::cout << "等级 " << static_cast<int>(grade) << " 人数: "
            << cnt << "\n";
    }

    // 附加任务:查找并打印
    int pos = findById(students, 1002);
    if (pos >= 0) std::cout << "学号 1002 是 " << students[pos].name << "\n";
    return 0;
}
```

设计说明:全程序零 `#define`、零 C 风格强转、零裸数组;先 `std::fixed` 再 `setprecision(2)` 保证两位小数。除法写 `sum / static_cast<double>(students.size())` 而非 `sum / students.size()`,是显式表达"我要浮点除法"——后者通常也能工作,但隐式转换意图不明,生产规范要求显式。

下一章预告:第2章 运算符与表达式——从 C 的运算符体系出发,讲清 C++ 的有符号溢出 UB、短路求值、优先级与求值顺序,以及 C++20 的三路比较运算符 `<=>`。